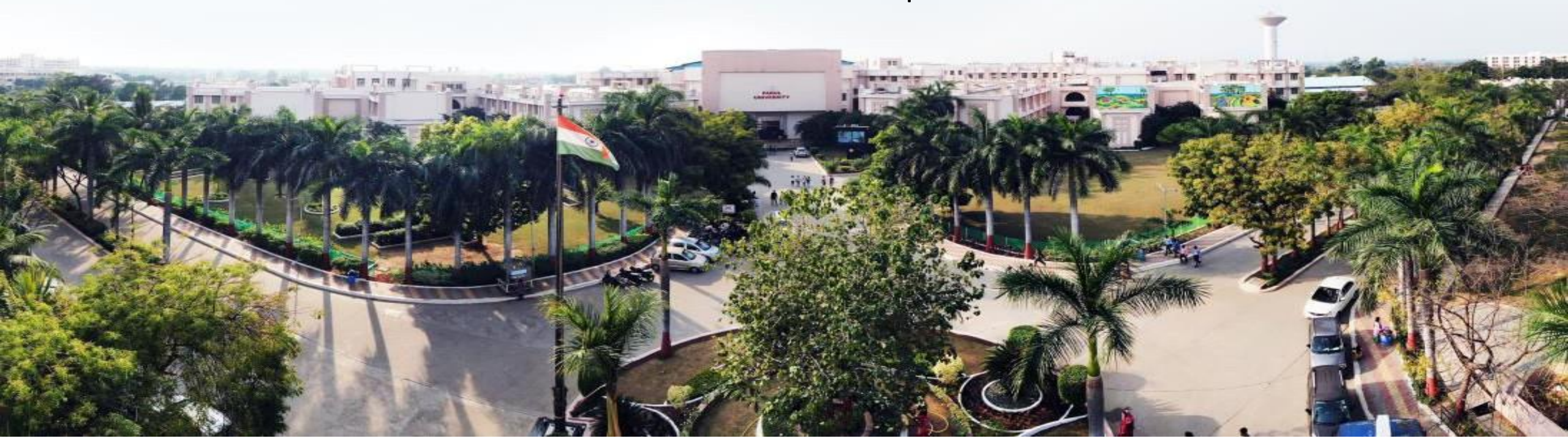




Design and analysis of Algorithms

Anand Jawdekar

Assistant Professor CSE Department



Unit-2

Divide and Conquer

Introduction

Divide and conquer is an algorithm design paradigm that works by recursively breaking down a problem into smaller, more manageable sub-problems until they become simple enough to solve directly. The solutions to the sub-problems are then combined to form a solution to the original problem. This technique is widely used in various computational problems, especially those involving large datasets or complex structures.

Key Steps in Divide and Conquer

Divide: Split the problem into several sub-problems that are smaller instances of the same problem.

Conquer: Solve the sub-problems recursively. If the sub-problems are small enough, solve them directly.

Combine: Merge the solutions of the sub-problems to form the solution to the original problem

Examples

Examples of Divide and Conquer Algorithms

Merge Sort:

Divide: Split the array into two halves.

Conquer: Recursively sort each half.

Combine: Merge the two sorted halves to produce the sorted array.

Quick Sort:

Divide: Choose a pivot element and partition the array into elements less than the pivot and elements greater than the pivot.

Conquer: Recursively sort the sub-arrays.

Combine: Concatenate the sorted sub-arrays and the pivot.

Binary Search:

Divide: Compare the target value to the middle element of the array.

Conquer: Recursively search in the left half if the target is smaller, or in the right half if the target is larger.

Combine: Return the position of the target value if found

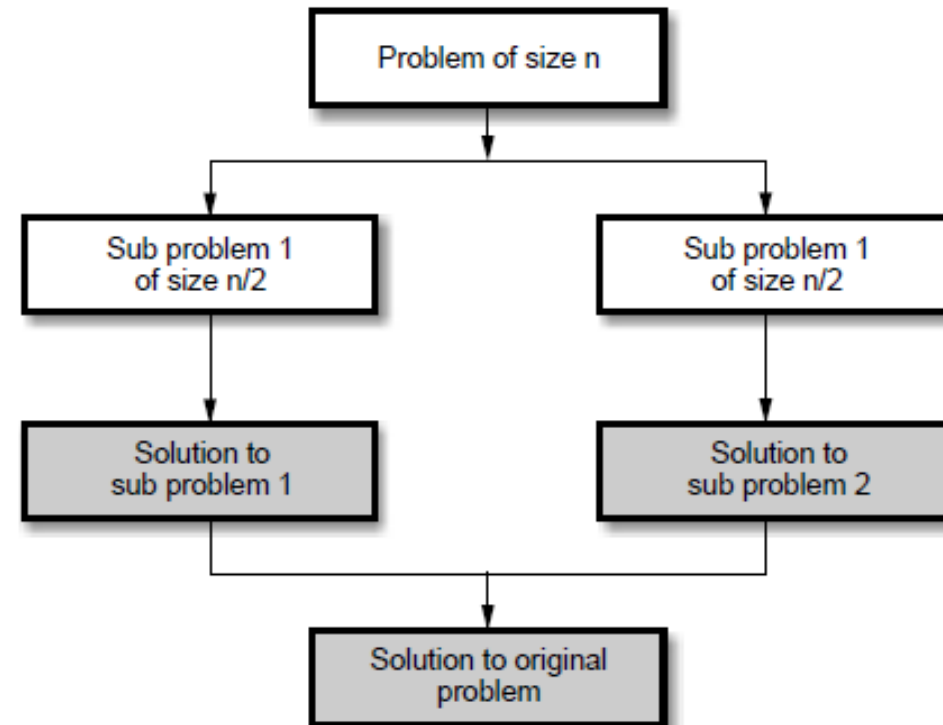
Applications of Divide and Conquer

- **Sorting algorithms** like merge sort and quicksort. **Search algorithms** like binary search.
- **Matrix multiplication** algorithms such as Strassen's algorithm.
- **Computational geometry** problems, including finding the closest pair of points.
- **Dynamic programming** problems that can be optimized using divide and conquer techniques..

Algorithm for Divide and Conquer

Algorithm DC(P)

```
{  
    if P is too small then  
        return solution of P.  
    else  
    {  
        Divide (P) and obtain  $P_1, P_2, \dots, P_n$   
  
        where  $n \geq 1$   
        Apply DC to each subproblem  
        return combine (DC( $P_1$ ), DC( $P_2$ ), ... (DC( $P_n$ )));  
    }  
}
```



Advantages and Disadvantages

Advantages:

Simplifies complex problems by breaking them down into simpler sub-problems.
Can be highly efficient in terms of time complexity, especially for sorting and searching algorithms.

Exploits parallelism since sub-problems can often be solved independently.

Disadvantages:

Overhead from recursive calls and the combination step can be significant.

Not always the most space-efficient due to additional memory usage for recursion and intermediate results.

May not be the best approach for problems where dividing the problem does not simplify it effectively.

Binary search

Binary search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing the search interval in half. Here's a step-by-step explanation:

Steps of Binary Search:

Initial Setup:

Start with two pointers: low at the beginning of the array and high at the end of the array.

Iteration:

Calculate the middle point: $\text{mid} = (\text{low} + \text{high}) / 2$.

Compare the middle element with the target value:

- If the middle element is equal to the target, the search is complete.

- If the target value is less than the middle element, set high to $\text{mid} - 1$.

- If the target value is greater than the middle element, set low to $\text{mid} + 1$.

Repeat:

Repeat the process until low exceeds high.

Termination:

- If the target value is found, return the index of the target value.

- If low exceeds high, the target value is not in the array.

Example

Example

Let's say we have a sorted array `arr = [1, 3, 5, 7, 9, 11]` and we want to find the target value 7.

Initial pointers: `low = 0`, `high = 5`.

Calculate `mid = (0 + 5) / 2 = 2.5`, which rounds down to 2. The element at index 2 is 5.

Since 7 is greater than 5, we set `low` to 3.

Calculate `mid = (3 + 5) / 2 = 4`. The element at index 4 is 9.

Since 7 is less than 9, we set `high` to 3.

Calculate `mid = (3 + 3) / 2 = 3`. The element at index 3 is 7.

The target value 7 is found at index 3.

Time Complexity Analysis

Best Case:

The best case occurs when the target value is found at the middle of the array in the first iteration. Time complexity: $O(1)$.

Worst Case and Average Case:

Both the worst-case and average-case scenarios occur when the target value is found at the last possible position or is not found at all, requiring the maximum number of iterations.

Each iteration of binary search cuts the search space in half.

If the array has n elements, the number of iterations needed can be determined by solving the equation $2^k = n$, where k is the number of iterations. Taking the logarithm (base 2) on both sides gives $k = \log_2(n)$

Therefore, the time complexity for both the worst-case and average-case scenarios is $O(\log n)$.

Quick sort

Quick sort is an efficient and widely used sorting algorithm that employs a divide-and-conquer strategy. Here's a detailed explanation of quick sort, including its steps, time complexity, and space complexity analysis.

Steps of Quick Sort

Choose a Pivot:

Select a pivot element from the array. The choice of pivot can vary (e.g., first element, last element, random element, or median).

Partitioning:

Rearrange the elements in the array so that all elements less than the pivot come before it, and all elements greater than the pivot come after it. The pivot is now in its final sorted position.

Recursively Apply:

Recursively apply the above steps to the sub-arrays formed by dividing the array around the pivot.

Example

Let's sort the array [3, 6, 8, 10, 1, 2, 1] using quick sort:

Choose a Pivot:

Let's choose the last element as the pivot, which is 1.

Partitioning:

Rearrange the array to [1, 1, 2, 3, 6, 8, 10] so that elements less than 1 are to its left and elements greater than 1 are to its right.

Recursively Apply:

Apply quick sort to sub-arrays [1] and [2, 3, 6, 8, 10].

Time Complexity Analysis

Best Case: $O(n \log n)$

Occurs when the pivot divides the array into two nearly equal halves each time.

Average Case: $O(n \log n)$

Occurs in a typical scenario where the pivot does not always perfectly divide the array but does so sufficiently well on average.

Worst Case: $O(n^2)$

Occurs when the pivot is the smallest or largest element every time, resulting in highly unbalanced partitions (e.g., already sorted array).

Merge sort

Merge sort is another efficient, stable, and comparison-based sorting algorithm that employs a divide-and-conquer strategy. Here's a detailed explanation of merge sort, including its steps, time complexity, and space complexity analysis.

Steps of Merge Sort

Divide:

Divide the unsorted array into two approximately equal halves until each sub-array contains a single element.

Conquer:

Recursively sort each sub-array.

Combine:

Merge the sorted sub-arrays to produce new sorted sub-arrays until there is only one sorted array remaining.

Example

Let's sort the array [38, 27, 43, 3, 9, 82, 10] using merge sort:

Divide:

Split the array into [38, 27, 43, 3] and [9, 82, 10].

Continue dividing: [38, 27] and [43, 3], [9, 82] and [10].

Further division: [38], [27], [43], [3], [9], [82], [10].

Conquer:

Merge [38] and [27] into [27, 38].

Merge [43] and [3] into [3, 43].

Merge [9] and [82] into [9, 82].

Combine:

Merge [27, 38] and [3, 43] into [3, 27, 38, 43].

Merge [9, 82] and [10] into [9, 10, 82].

Finally, merge [3, 27, 38, 43] and [9, 10, 82] into [3, 9, 10, 27, 38, 43, 82].

Time Complexity Analysis

Best Case: $O(n \log n)$ Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Merge sort consistently divides the array into two halves and merges them back together in $O(n \log n)$ time, regardless of the initial order of the elements.

Strassen's matrix multiplication

Strassen's algorithm optimizes the multiplication of two matrices by reducing the number of required multiplications, which are generally more computationally expensive than additions. Strassen's matrix multiplication is an efficient algorithm to multiply two matrices faster than the conventional method. Here's a brief overview:

Basics

Traditional Matrix Multiplication: Requires $O(n^3)$ operations for multiplying two $n \times n$ matrices.

Strassen's Algorithm: Reduces the number of operations to approximately $O(n^{2.81})$.

How It Works

Strassen's method divides each matrix into four submatrices. For matrices AAA and BBB, this looks like:

$$A = \begin{bmatrix} A_{11} & \cdots & A_{12} \\ \vdots & \ddots & \vdots \\ A_{21} & \cdots & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & \cdots & B_{12} \\ \vdots & \ddots & \vdots \\ B_{21} & \cdots & B_{22} \end{bmatrix}$$

Instead of performing 8 multiplications, Strassen's algorithm uses 7:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Cont..

Combining Results

The product matrix C is then computed as:

$$C = \begin{bmatrix} c_{11} & \cdots & c_{12} \\ \vdots & \ddots & \vdots \\ c_{21} & \cdots & c_{22} \end{bmatrix}$$

where:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

Complexity Analysis

Traditional Matrix Multiplication

Given matrices:

A of size $n \times m$

B of size $m \times p$

Steps Involved

Initialize Result Matrix C:

Size: $n \times p$

Initial elements: zero

Compute Each Element in C

$$C[i][j] = \sum_{k=1}^m A[i][k] \times B[k][j]$$

rows of A.

columns of B.

n elements (columns of A and rows of B).

and Additions:

Total Operations for Entire Matrix: $n \cdot p \cdot m$

Cont..

Time Complexity

Overall Time Complexity:

$O(n \times m \times p)$

Special Case

Square Matrices ($n=m=p$)

Complexity simplifies to $O(n^3)$

Space Complexity: $O(n \times p)$ for storing result matrix C.

Strassen's Algorithm Complexity Analysis

Strassen's method multiplies two $n \times n$ $\times$ $n \times n$ matrices by dividing them into smaller submatrices and performing fewer multiplications.

Steps

Divide Matrices:

Split each $n \times n$ $\times$ $n \times n$ matrix into four $\frac{n}{2} \times \frac{n}{2} \times \frac{n}{2} \times \frac{n}{2}$ submatrices.

Compute 7 Products

Instead of 8, Strassen's uses 7 matrix multiplications for these submatrices:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Combine Products:

Compute the resulting submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ using additions and subtractions of these products.

Cont..

Complexity Analysis

Recursive Formula:

For each recursive level, the time complexity $T(n)$ is:

$$T(n) = 7T(n/2) + O(n^2)$$

The $O(n^2)$ term accounts for matrix addition and subtraction.

Master Theorem Application:

Using the Master Theorem for divide-and-conquer recurrences:

$$T(n) = aT(n/b) + f(n)$$

Here, $a=7$, $b=2$ and $f(n)=O(n^2)$.

Solution:

$$T(n) = O(n^{\log_2 7}) \approx 2.81$$

Max-Min Algorithm using Divide and Conquer

A max-min problem typically involves finding the maximum of the minimum values in a set of elements. Here's a simple example:

You have several groups of numbers, and you want to determine the maximum of the minimum values from each group.

Steps to Solve

Identify the Groups: Divide your numbers into distinct groups.

Find the Minimum of Each Group: Determine the smallest number in each group.

Find the Maximum of These Minimums: Compare the minimums from each group to find the largest among them.

Example

Consider an array of numbers: [3, 5, 1, 8, 2, 9, 4].

Solution

Divide the array: Left half: [3, 5, 1] Right half: [8, 2, 9, 4]

Conquer each half recursively:

Left half ([3, 5, 1]):

Divide into [3] and [5, 1]

Find max-min of [3]: (3, 3)

Find max-min of [5, 1]: (5, 1)

Combine results: Max = 5, Min = 1

Right half ([8, 2, 9, 4]):

Divide into [8, 2] and [9, 4]

Find max-min of [8, 2]: (8, 2)

Find max-min of [9, 4]: (9, 4)

Combine results: Max = 9, Min = 2

Combine results from both halves:

Overall max: $\max(5, 9) = 9$

Overall min: $\min(1, 2) = 1$

Algorithm

```
function findMaxMin(arr, low, high):  
    if low == high:  
        return (arr[low], arr[low]) // Only one element  
  
    if high == low + 1:  
        if arr[low] > arr[high]:  
            return (arr[low], arr[high])  
        else:  
            return (arr[high], arr[low])  
        mid = (low + high) // 2  
    (max1, min1) = findMaxMin(arr, low, mid)  
    (max2, min2) = findMaxMin(arr, mid + 1, high)  
  
    overall_max = max(max1, max2)  
    overall_min = min(min1, min2)  
  
    return (overall_max, overall_min)
```

Explanation

Base Cases:

If the subarray has one element, that element is both the max and min.

If the subarray has two elements, compare them to determine the max and min directly.

Recursive Division:

Split the array into two halves.

Recursively determine the max and min for each half.

Combine Results:

Compare the maximums of the two halves to get the overall maximum.

Compare the minimums of the two halves to get the overall minimum.

Complexity

Time Complexity: $O(n)$, where n is the number of elements. This is because each element is compared a constant number of times.

Space Complexity: $O(\log n)$ due to the recursive call stack.

Advantages

Efficiency: The divide-and-conquer approach minimizes the number of comparisons needed compared to a straightforward linear scan.

Scalability: It works well for large datasets due to its logarithmic recursive depth.

PU